Maritime Tanker Tracker Technical Summary

A Real-Time AIS Data Pipeline for Cargo-Handling Event Detection and Vessel Payload Estimation

Program Developed and Report Written by Joseph Jason

Abstract

This technical summary outlines a modular pipeline, developed in R, that ingests live global Automatic Identification System (AIS) data, identifies tanker vessels from the raw stream, uses deterministic filtration alongside DBSCAN to clean data, and applies a Guassian Mixture Model (GMM), alongside hydrostatic and ballast-incorporating models to detect cargo-handling events and estimate the mass of cargo transferred. The system is implemented across four concurrent processes and draws on methods from Zhang et al. (2025), Papanikolaou (2014), Kalokairinos et al. (2000-2005), Schneekluth and Bertram (2003), MARPOL 73/78, and Prakash and Smith (2019), as well a self-derived deadweight tonnage regression model.

1. Introduction

The global movement of crude oil and refined petroleum products is among the most economically significant flows in international trade. Tanker vessels carry the majority of this cargo, and their loading and unloading behaviour at ports and offshore terminals provides a real-time signal of supply, demand, and inventory conditions that is not otherwise directly observable. Automatic Identification System (AIS) transponders are mandatory on vessels above 300 gross tonnes and transmit position, speed, and static vessel data at regular intervals, creating a continuous global feed of maritime movement data (IMO, 2015). When properly processed, AIS data can be used to infer when, where, and how much cargo a tanker transferred.

This paper documents a data pipeline that was designed and implemented to do precisely that. The system collects a global AIS stream via WebSocket, isolates tanker messages at the hexadecimal level before full parsing, cleans the resulting data using Zhang et al.'s (2025) filters and Density-Based Spatial Clustering of Applications with Noise (DBSCAN), detects significant changes in reported draught as proxies for cargo-handling events, fits a Gaussian Mixture Model (GMM) to speed-over-ground data to identify true berthing events within those draught-change windows, and applies a chain of hydrostatic formulas and regression estimates to quantify the mass of cargo transferred, while accounting for change in the mass of ballast.

The pipeline is structured across 4 concurrent processes that communicate through the filesystem. Process 1 connects to AISStream.io via WebSocket and buffers raw hexadecimal messages to disk. Process 2 pre-filters messages at the hexadecimal level, parses only tanker messages, deduplicates the result, and writes structured RDS files for downstream consumption. Process 3 loads the accumulated filtered data, applying the full analytical pipeline — cleaning, draught-change detection, cargo quantification, port assignment, GMM berthing detection, and vessel classification — and appending confirmed events to a local master output file. Process 4 saves the process 2 output into a MySQL database for data preservation.

The methodological foundation is Zhang et al. (2025), whose AIS-driven framework for cargo-handling event identification in international trade tankers is adapted and extended here. Extensions include the removal of the course-over-ground filter, the easing of temporal and spatial thresholds to accommodate the data quality limitations of a single terrestrial AIS feed, an author-derived DWT regression estimated from previously collected AIS data, a lightship and ballast accounting procedure drawing on a Marine

Administrationn (MARAD) table in Papanikolaou (2014, p. 548), MARPOL 73/78, and a consideration of the Prakash and Smith payload estimation approach (2019) as an alternative framework.

Before proceeding to the rest of this technical summary, note

2. Literature Review

Zhang et al. (2025) provide the primary methodological foundation for this system. Their paper, “AIS data-driven analysis for identifying cargo handling events in international trade tankers” (Ocean Engineering, Vol. 317), proposes a multi-stage pipeline that uses static AIS reports for draught changes and dynamic position reports for movement characterisation. Their method applies DBSCAN to remove draught outliers within spatially proximate observations, filters draught-change events by thresholds on magnitude (>1 m), elapsed time (>12 h), and travelled distance (>12 km), and uses a GMM fitted to speed-over-ground data to distinguish berthed observations from underway observations within each candidate event window. This system implements that core logic while adapting several design choices to the constraints of the present data environment, as detailed in subsequent sections.

Papanikolaou (2014), Ship Design: Methodologies of Preliminary Design (Springer), provides two regression equations used here. The first estimates lightship tonnage from displacement. The second provides the MARAD vessel dimension coefficient table, which maps length-to-beam and beam-to-draught ratios to block coefficients (Cb) and midship section coefficients (Cm), both of which are required inputs to the waterplane area coefficient and the tonnes-per-centimetre immersion (TPCI) formulas that underpin cargo mass estimation.

Schneekluth and Bertram (2003), Ship Design for Efficiency and Economy, provide the formula for the waterplane area coefficient (Cw) in terms of Cb and Cm: $Cw = (1 + 2(Cb/Cm)) / 3$. This coefficient scales the plan-view area of the waterplane — the portion of the hull cross-section at the waterline — and is a central input to TPCI.

Kalokairinos et al. (2000-2005) contribute a regression model for design draught as a function of deadweight tonnage, found in Papanikolaou (2014, p. 474): design draught = $0.45011 \times DWT^{0.303134}$. Design draught is the draught when a vessel is loaded to its intended, optimal operating condition and, unlike reported draught, is a fixed physical characteristic of each ship. It enters the displacement calculation and, through displacement, the lightship tonnage estimate.

MARPOL 73/78, Annex I, Regulation 18 (the MARPOL Training Institute edition) specifies the minimum light ballast condition for tankers: the minimum ballast draught in metres is $2.0 + 0.02 \times L$, where L is the vessel’s registered length. This regulation exists to ensure that the propeller and rudder remain sufficiently submerged for safe maneuvering in ballast condition. It is used here to estimate the light ballast draught and, combined with TPCI and lightship tonnage, the mass of light ballast in metric tonnes.

The vessel classification lookup, mapping DWT ranges to commercial vessel classes (Small Tanker through ULCC), is drawn from the U.S. Energy Information Administration and porteconomicsmanagement.org. The conversion between metric tonnes and barrels of oil uses the standard API factor of 7.3 barrels per metric tonne.

3. Process 1 - WebSocket Ingestion

3.1 Data Selection

Process 1 is the entry point of the pipeline. It subscribes to the AISStream.io WebSocket feed and collects two AIS message types: PositionReport (dynamic data: latitude, longitude, speed over ground, course over ground, rate of turn, and navigational status) and ShipStaticData (static data: MMSI, IMO number, ship name, vessel type code, maximum static draught, and antenna-derived dimensions). The subscription uses a global

bounding box — latitude [–90, 90], longitude [–180, 180] — deliberately broad so as not to restrict collection by geographic zone at the ingestion stage:

```
sub_msg <- list(
    APIKey = ais_api_key,
    # Global bounding box - no need to restrict non-ports if fast enough
    BoundingBoxes = list(list(list(-90, -180), list(90, 180))),
    FilterMessageTypes = list("PositionReport", "ShipStaticData")
  )
```

The static report provides draught — the vertical distance from the waterline to the keel — which is the primary signal used downstream to infer cargo loading and unloading events, as noted by Zhang et al. (2025). The position report provides the spatial and kinematic context needed to characterize vessel behaviour within each candidate event window.

3.2 WebSocket Connection and Reconnection Logic

To minimize latency, all relevant actions are created as functions and loaded prior to the WebSocket connection loop that iteratively attempts to connect. Upon opening, the socket must receive a subscription message within three seconds. The main event loop checks for connection state at each iteration. If the socket is closed (readyState = 3) or null, the process attempts reconnection with exponential backoff, starting at a one-second delay and capping at 60 seconds. A 30-second timeout is also enforced: if no message is received within 30 seconds of the last message, the socket is forcibly closed and a reconnection is initiated. This guards against silent connection failures where the socket appears open but is no longer receiving data. This allows this program to run continuously, and has even succeeded when left to run locally for days at a time.

3.3 Buffer and Flush

Received messages arrive as raw bytes. If the message is of type raw, individual bytes are converted to a hex string before buffering. Strings accumulate in an in-memory character vector. A flush is triggered when either 50,000 messages have accumulated or 15 seconds have elapsed since the last flush. On flush, the buffer is written as a plain-text file with a timestamped and sequentially numbered name (raw_hex_YYYYMMDD_NNN.txt), the buffer is cleared, and garbage collection is invoked explicitly. The file counter increments so each flush produces a unique file. The process continues until either a kill-switch file (STOP_AIS.txt) is detected or a Ctrl+C interrupt is received.

4. Process 2 - Parser and Filter

4.1 Rationale for Hexadecimal Pre-Filtering

AIS data arrives as a stream of JSON-encoded messages, which Process 1 stores in their hexadecimal representation rather than decoding. The key design decision in Process 2 is to filter messages at the hexadecimal level before performing any full JSON deserialisation. This is motivated by the computational cost of parsing: deserialising every message to a JSON object, only to discard the majority, was found to be too expensive to sustain a stable WebSocket connection while simultaneously processing the stream.

The solution is to define fixed hexadecimal patterns for the substrings that identify message type and vessel class. Because the AIS protocol fixes the structure of JSON field names, their hexadecimal encodings are invariant:

```
HEX_STATIC   <- "225368697053746174696344617461"  # "ShipStaticData"
HEX_POSITION <- "22506f736974696f6e5265706f7274"  # "PositionReport"
HEX_TANKER_PATTERN_START <- "2254797065223a38"     # '"Type":8'
```

AIS ship type codes 80–89 denote tankers. Because the hex encoding of ‘8’ followed by ASCII digits 0–9 is predictable (‘30’ through ‘39’), a vessel can be identified as a tanker without parsing the message at all:

```
is_tanker_hex <- function(hex_string) {
  type_pos <- regexpr(HEX_TANKER_PATTERN_START, hex_string, fixed = TRUE)
  if (type_pos == -1) return(FALSE)
  second_digit_hex <- substr(hex_string, type_pos + nchar(HEX_TANKER_PATTERN_START),
                             type_pos + nchar(HEX_TANKER_PATTERN_START) + 1)
  return(second_digit_hex %in% c("30","31","32","33","34",
                                 "35","36","37","38","39"))
}
```

This avoids deserialising the large majority of the stream. The ratio of hex-filtered messages to actually-parsed messages is tracked as a speedup metric and reported at the end of each processing run.

4.2 MMSI Registry

Once a vessel has been identified as a tanker from its static message, its MMSI is added to an in-memory set and persisted to an RDS file. Subsequent dynamic (position) messages are filtered by checking whether their MMSI appears in this registry — again at the hexadecimal level, before parsing. This means that position reports for non-tanker vessels are discarded without being deserialised at all. The registry is saved on shutdown and reloaded on restart, so accumulated tanker identities persist across sessions.

It should be noted that the “in-memory set” used is an R environment variable, which was chosen because it performs similarly to a hash table and providing O(1) time-complexity, regardless of length.

4.3 Field Extraction and Parsing

For messages that pass hexadecimal pre-filtering, the extract_hex_value function reads a numeric or string value directly from the raw hex by locating a field marker, advancing past it, and reading hex pairs until a delimiter (comma, quote, brace, or period) is reached. Only after this targeted extraction has confirmed a message’s relevance is the full hex_to_json_full deserialisation called. Parsed static records contain: MMSI, time (UTC), ship name, IMO number, vessel type, draught, destination, and length and breadth derived from antenna offset dimensions A+B (length) and C+D (breadth). Parsed dynamic records contain: MMSI, time, ship name, latitude, longitude, SOG, COG, rate of turn, and navigational status.

4.4 Data Deduplication

Dynamic messages are deduplicated within each processing batch by retaining the last observation per MMSI per minute. This prevents the double-counting of position reports that may have been transmitted multiple times within a single AIS update interval:

```
dynamic_list[, time_minute := as.POSIXct(format(time_utc, "%Y-%m-%d %H:%M:%S"), tz="U
TC")]
setorder(dynamic_list, mmsi, time_minute, -time_utc)  # latest first
dynamic_list <- dynamic_list[, .SD[1], by = .(mmsi, time_minute)]
```

The process runs as a continuous loop, checking for new hex files every 10 seconds. Files younger than 5

seconds are skipped to avoid reading files that Process 1 is still writing, which would otherwise create a race condition. Processed hex files are deleted on success. Files that encounter errors are retained with a log entry for manual inspection. Statistics — messages processed, tankers identified, dynamic messages kept versus filtered, duplicates removed, and parse errors — are printed after each file.

5. Process 3 - Deterministic Filtration, Event Detection, and Payload Esimation

Process 3 is the analytical core of the system. It runs on a 24-hour cycle and applies the full Zhang et al. (2025) pipeline to all accumulated filtered data, extended with ballast accounting and vessel classification. It is divided here into three logical stages: data cleaning (5.1–5.2), cargo-handling event identification (5.3–5.4), and vessel payload estimation (5.5–5.7).

5.1 Data Loading and Standardization

At each cycle, the process loads all filtered_static*_.rds and filtered_dynamic*_.rds files produced by Process 2. A column standardization step normalizes variant field names (e.g., beam_m and width_m are both renamed to breadth_m) that may have arisen from changes in parsing conventions across the life of the system. Files are combined with rbindlist into two master data.tables, one static and one dynamic. An overlap window of 60 hours is maintained by archiving input files only after that duration has elapsed. Because Process 3 runs every 24 hours, this guarantees that events whose draught-change window straddles a cycle boundary — the first observation falling in one batch, the second in the next — are captured in the subsequent cycle rather than being silently lost. Although 60 hours will require excessive compute because the same events will be calculated multiple times, this allows excessively long loading/unloading events to be observed.

5.2 Deterministic and Statistical Cleaning

5.2.1 Static Data Cleaning

Static records are first filtered deterministically: any record with a missing or non-positive draught, missing MMSI, or missing coordinates or dimensions is rejected. This eliminates structurally invalid records before statistical methods are applied.
DBSCAN is then applied to the draught values for each MMSI individually, using an epsilon of 2 meters and a minimum cluster size of 2. This follows the Zhang et al. (2025) recommendation to use DBSCAN's noise-labeling property to remove draught outliers.

```
static_clean <- static[, {
  if (.N > 2) {
    db_result <- dbscan(as.matrix(.SD$draught), eps = 2, minPts = 2)
    .SD[db_result$cluster != 0]  # cluster 0 = noise
  } else { .SD }
}, by = mmsi]
```

5.2.2 Dynamic Data Cleaning

Dynamic records are filtered by physical plausibility bounds: latitude must be in [–90, 90], longitude in [–180, 180], SOG in [0, 40] knots, COG in [0, 360] degrees. Records failing any condition are rejected.

```
# Clean dynamic data (remove invalid coordinates and speeds)
dynamic <- dynamic[!is.na(lat) &
                       !is.na(lon) &
                       !is.na(sog) &
                       lat >= -90 &
                       lat <= 90 &
                       lon >= -180 &
                       lon <= 180 &
                       sog >= 0 &
                       sog <= 40 &
                       !is.na(mmsi) &
                       !is.na(cog) &
                       cog >= 0 &
                       cog <= 360, ]

  # Clean cog data - shelve it for now
  minute_threshold <- 6
  dynamic[, cog := {
    time_diff_prev <- as.numeric(difftime(time_utc, shift(time_utc, 1), units = "min
s"))
    time_diff_next <- as.numeric(difftime(shift(time_utc, -1), time_utc, units = "min
s"))
    cog_diff_prev <- abs((cog - shift(cog, 1) + 180) %% 360 - 180)
    cog_diff_next <- abs((cog - shift(cog, -1) + 180) %% 360 - 180)
    # If a large change in cog occurs over a short time
    fifelse(time_diff_prev < minute_threshold &
              time_diff_next < minute_threshold &
              (cog_diff_prev > 30 & cog_diff_next > 30),
            # Replace with average of adjacent trajectory points
            (shift(cog, 1) + shift(cog, -1))/2,
            # Else cog
            cog)
  }, by = mmsi]
```

5.3 Cargo-Handling Event Identification

5.3.1 Draught Change Detection and Threshold Design

Cargo-handling events are identified from the static data stream by detecting significant changes in reported draught for each vessel over time. The static data is sorted by MMSI and time-stamp, and records where draught does not change from one observation to the next are removed (retaining only records adjacent to a change). For each remaining record, the draught difference, elapsed time, and haversine distance from the previous observation are calculated:

```
static[, change := draught != shift(draught, 1), by = mmsi]
static <- static[change == TRUE | shift(change, 1, type = "lead") == TRUE]
static[, draught_diff   := draught - shift(draught, 1), by = mmsi]
static[, time_diff_hours := (date_seconds - prev_date_seconds) / 3600]
static[valid_coords_idx, distance_km := distHaversine(coords_prev, coords_curr) / 100
0]
events <- static[abs(draught_diff) > 1,
                 time_diff_hours > 12,
                 distance_km > 12]
```

5.3.2 Port Assignment

Dynamic observations are matched to named ports and terminals using a set of hand-coded bounding boxes covering 61 locations across seven geographic groups: global chokepoints (Fujairah, Singapore, Suez, Istanbul, etc.), offshore supply terminals in the Middle East and West Africa, offshore discharge terminals in China, India, and the U.S. Gulf, European and Mediterranean ports, Asia-Pacific and Latin American terminals, Russian export terminals, and the Strait of Hormuz gate. A non-equi join matches each dynamic observation to any bounding box that contains its latitude and longitude. Observations that fall outside all defined boxes are labelled “Lost at Sea”.

5.3.3 GMM-Based Berthing Detection

For each candidate draught-change event (identified by MMSI and the timestamp of the static report), the dynamic trajectory falling within the event’s time window — between the previous draught observation and the current one — is extracted and passed to the find_berthing_event function. This function fits a two-component Gaussian Mixture Model to the SOG values in that segment, following Zhang et al. (2025):

```
model <- Mclust(local_segment$sog, G = 2, verbose = FALSE)
means  <- model$parameters$mean
moored_mean_id  <- which.min(means)  # moored/berthing component
threshold_sog   <- means[moored_mean_id] +
                   3 * sqrt(model$parameters$variance$sigmasq[moored_mean_id])
```

The GMM separates the SOG distribution into a low-speed (moored or anchored) component and a high-speed (underway) component. The berthing threshold is set at the mean of the moored component plus three standard deviations, capturing approximately 99.7% of the moored distribution under the normality assumption. This threshold is intentionally conservative in one direction: by setting it at mean + 3σ rather than a lower multiple, it errs on the side of classifying more observations as moored rather than underway, which reduces Type I errors (labelling genuinely moored observations as moving) at the cost of increasing Type II errors (classifying some slow-steaming or manoeuvring observations as moored).

The event is confirmed as a true berthing event if at least two dynamic observations fall below the threshold SOG. If confirmed, the berthing observations are returned with their timestamps, coordinates, and port labels. If the segment has fewer than three observations, fewer than two unique SOG values, or zero variance in SOG, all conditions under which the GMM cannot be meaningfully fitted, then the event is tagged as missing a berthing confirmation and retained in a separate diagnostics table.

5.4 Deadweight Tonnage and Design Draught Estimation

Estimating the mass of cargo transferred requires knowledge of a vessel’s physical characteristics beyond what AIS alone provides. The most critical of these is deadweight tonnage (DWT) — the total mass a vessel can carry including cargo, fuel, crew, and provisions — which is not transmitted in AIS messages. The

approach here is to estimate DWT from vessel length using a regression fitted from a previous collection of AIS data. Note that this data set is relatively small because obtaining additional data is a relatively intensive process.

```
events[, est_dwt := -971.2256 + 1.0273 * length_m * breadth_m * draught]
```

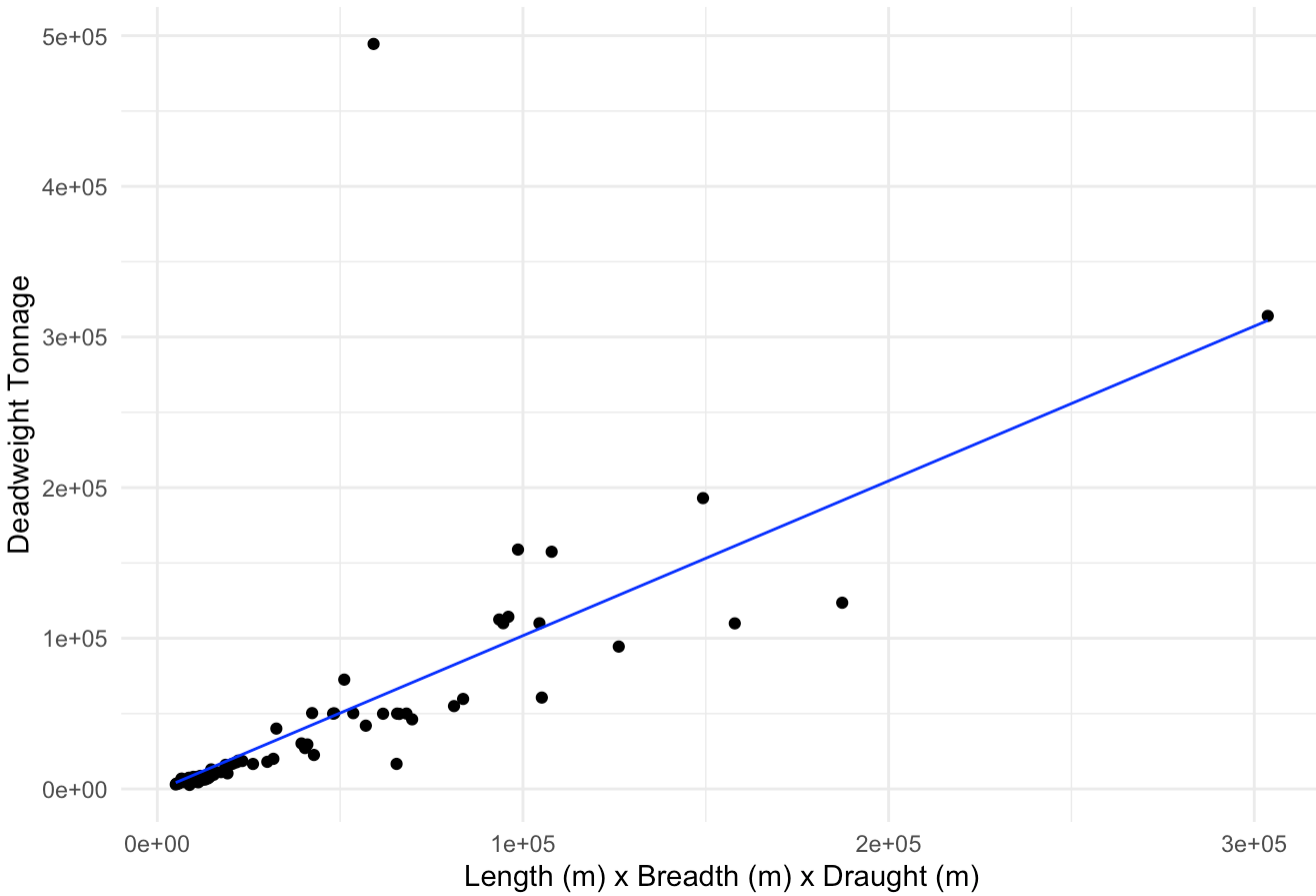

Table 1. OLS Regression Results (DWT ~ factor)

Term	Estimate	Std. Error	t value	p value
(Intercept)	-971.2256	7018.9374	-0.138	0.890
factor	1.0273	0.1126	9.128	2.98e-14 ***

Signif. codes: 0 = *** 0.001 = ** 0.01 = * 0.05 = . 0.1 = ' '. Residual standard error: 50,420. Degrees of freedom: 85.
Multiple R-squared: 0.495. Adjusted R-squared: 0.489. F-statistic: 83.31. DF: 1 and 85. p-value: 2.979e-14

This regression was derived by the author from a dataset of vessel specifications obtained from industry sources, with the product of length, breadth, and draught as being, as a predictor of DWT, statistically significant at the 0.001 level, having a p-value of 2.98e-14. It is noted in the codebase that this is an area of ongoing refinement. A planned improvement is to replace the single regression with a decision tree or cluster-

then-regress approach, using the length-to-beam ratio and beam-to-draught ratio as additional predictors, which should improve accuracy for vessels at the extremes of the size distribution. However, for the time being, this paper then estimates design draught from DWT using the Kalokairinos et al. regression, found in Papanikolaou's "Ship Design" (2014, p. 474):

```
events[, design_draught := 0.45011 * (est_dwt ^ 0.303134)]
```

5.5 Hull Coefficients and TPCI

The waterplane area coefficient and tonnes-per-centimetre immersion are derived from the MARAD vessel dimension lookup table, which maps length-to-beam and beam-to-draught ratios (computed from each vessel's reported dimensions and estimated design draught) to block and midship coefficients.

```
events[, length_beam := round(length_m / breadth_m * 2) / 2]  # 0.5 increments
events[, beam_draught := round(breadth_m / design_draught * 4) / 4]  # 0.25 increment
s
```

The ratios are rounded to the nearest grid increment before joining to the MARAD table, with a roll-to-nearest lookup (roll = "nearest") ensuring that vessels whose ratios fall between tabulated values receive the closest match. Missing matches default to Cb = 0.85 and Cm = 0.994 — values characteristic of a typical loaded tanker.

```
events <- MARAD[events,
  .(... block_c = fcoalesce(block_c, 0.850),
        middle_c = fcoalesce(middle_c, 0.994)),
  on = .(length_beam, beam_draught), roll = "nearest", mult = "first"]
```

The waterplane area coefficient and TPCI are then computed by comparing tanker values against the table below to derive block and middle coefficients, and subsequently using Schneekluth's and Bertram's waterplane coefficient from "Ship Design for Efficiency and Economy" (p. 11).

```
events[, waterplane_c := (1 + 2 * (block_c / middle_c)) / 3]
events[, tpci         := length_m * breadth_m * waterplane_c * 1.025 / 100]
```

Table 2. MARAD Vessel Dimension Coefficients (Papanikolaou, 2014)

Parameter	Length/Beam	Beam/Draught	Middle Coeff. (Cm)	Block Coeff. (Cb)
A	5.5	3.00	0.994	0.875
B	6.0	3.00	0.994	0.875
C	6.5	3.00	0.994	0.875
D	4.5	3.00	0.994	0.850
E	5.0	3.00	0.994	0.850
F	5.5	3.00	0.994	0.850
G	5.0	3.00	0.994	0.800

Parameter	Length/Beam	Beam/Draught	Middle Coeff. (Cm)	Block Coeff. (Cb)
H	6.5	3.00	0.994	0.850
I	6.0	3.75	0.994	0.850
J	6.0	4.50	0.994	0.850
K	5.0	3.75	0.994	0.850
L	5.0	4.50	0.994	0.850
M	5.5	3.75	0.994	0.875
N	5.0	3.75	0.994	0.800
O	5.5	3.75	0.994	0.875
P	5.0	4.50	0.994	0.800

5.6 Cargo Mass Estimation Incuding Ballast Accounting

5.6.1 DWT Change and Observed Mass

The raw estimate of cargo transferred is the change in displacement inferred from the change in draught. TPCI converts draught change (in centimetres) to mass (in metric tonnes):

```
events[, dwt_change := tpci * draught_diff * 100]
events[, abs_mass   := abs(dwt_change)]
```

This quantity represents the net change in the mass of everything aboard the vessel — cargo, ballast, fuel, and stores. To isolate cargo, it is necessary to account for the simultaneous change in ballast, which moves in the opposite direction to cargo: tankers pump in ballast water when unloading cargo to maintain stability and trim, and pump it out when loading.

5.6.2 Ballast Accounting

Ballast accounting follows a four-step procedure. First, displacement at design draught is calculated, treating the vessel as if fully loaded.

```
events[, displacement := length_m * breadth_m * design_draught * block_c * 1.025]
```

Second, lightship tonnage — the mass of the empty vessel, including its permanent structure, machinery, and equipment but excluding all variable loads — is estimated from the Papanikolaou (2014) regression.

```
events[, lightship_tonnage := 2.9186 * (displacement ^ 0.75548)]
```

Third, light ballast draught is estimated from the MARPOL 73/78, Annex I, Regulation 18 formula, and converted to mass using TPCI and lightship tonnage.

```
events[, light_ballast_m     := 2.0 + 0.02 * length_m]
events[, light_ballast_tonnes := (tpci * 100 * light_ballast_m) - lightship_tonnage]
```

Fourth, heavy ballast (the ballast carried when a vessel is fully or partially laden) is estimated at 5% of DWT,

consistent with standard industry practices:

```
events[, heavy_ballast_tonnes := est_dwt * 0.05]
```

The change in ballast is then the difference between light and heavy ballast, and cargo transferred is the observed mass change plus the ballast change:

```
events[, ballast_change_t    := light_ballast_tonnes - heavy_ballast_tonnes]
events[, cargo_tonnes_change := abs_mass + ballast_change_t]
events[, cargo_barrels_change := cargo_tonnes_change * 7.3]
```

The direction of the event (loading or unloading) is inferred from the sign of the draught difference: a positive draught change indicates loading (the vessel sits deeper); a negative change indicates unloading.

Table 3. Derived Variable Equations and Sources

Variable	Equation	Source
est_dwt (t)	`exp(-5.7435 + 3.1366 × ln(length_m))`	Author regression (industry data)
design_draught (m)	`0.45011 × est_dwt^0.303134`	Kalokairinos et al.
waterplane_c	`(1 + 2 × (Cb / Cm)) / 3`	Schneekluth & Bertram (2003)
tpci (t/cm)	`L × B × Cw × 1.025 / 100`	Standard naval architecture
dwt_change (t)	`tpci × draught_diff × 100`	Zhang et al. (2025)
displacement (t)	`L × B × d_design × Cb × 1.025`	Naval architecture
lightship_tonnage (t)	`2.9186 × displacement^0.75548`	Papanikolaou (2014)
light_ballast_m (m)	`2.0 + 0.02 × L`	MARPOL 73/78, Annex I, Reg. 18
light_ballast_t (t)	`(tpci × 100 × light_ballast_m) - lightship_t`	Derived
heavy_ballast_t (t)	`est_dwt × 0.05`	Industry standard
ballast_change (t)	`light_ballast_t - heavy_ballast_t`	Derived
cargo_change (t)	`abs(dwt_change) + ballast_change`	Zhang et al. (2025) + ballast adjustment
cargo_change (bbl)	`cargo_change_t × 7.3`	API conversion

5.6.3 Relationship to Prakash and Smith

The ballast accounting procedure above is conceptually related to, but methodologically distinct from, the approach proposed by Prakash and Smith. Their framework uses the operational block coefficient to estimate payload directly from a displacement difference, without separately modelling ballast. Their Equation 7 regression further corrects the estimate using lightship weight. The key advantage of their approach is that it computes payload as a function of total draught (rather than draught change), making it less sensitive to noise in individual AIS reports. This, however, is more difficult to attain strictly from AIS data.

5.7 Vessel Classification

Confirmed cargo events are classified by vessel size using the DWT-based lookup table drawn from EIA and porteconomicsmanagement.org data. Events whose absolute mass falls outside any defined DWT range are classified as “Unclassified”. Each confirmed event in the output table carries the commercial class, technical class, and inferred primary cargo type for the vessel.

Table 4. Vessel Classification Lookup (EIA; porteconomicsmanagement.org)

DWT Range (t)	Commercial Class	Code	Primary Cargo
< 10,000	Small Tanker	ST	Refined
10,000 – 25,000	Handysize	HS	Refined
25,000 – 45,000	Handymax	HM	Refined / Crude
45,000 – 80,000	Panamax	PM	Refined / Crude
80,000 – 120,000	Aframax	AF	Crude
120,000 – 200,000	Suezmax	SZ	Crude
200,000 – 320,000	VLCC	VLCC	Crude
>= 320,000	ULCC	ULCC	Crude

6. Output Structure and Aggregation

Each 24-hour cycle appends confirmed events to a master file (data/output/all_cargo_events.rds). Deduplication before appending rounds event start times to 4-hour blocks and drops records sharing an MMSI and time block, preventing the double-counting of events that appear in overlapping batch windows. Each confirmed event record contains:

- Vessel identifiers: MMSI, IMO, vessel name
- Event classification: event type (loading/unloading), commercial class, technical class, primary cargo
- Location: port name, start and end latitude/longitude
- Timing: event start time and event end time
- Physical characteristics: length, breadth, design draught, displacement, DWT, lightship tonnage, waterplane coefficient, TPCI
- Mass estimates: draught change (m), estimated mass (t), absolute mass (t), light ballast (m and t), heavy ballast (t), ballast change (t), cargo change (t and bbl)

Intermediate files — filtered RDS files from Process 2 — are archived after 60 hours. Processed output files from individual cycles are deleted after successful aggregation, retaining only the most recent cycle file alongside the master. All processes share a kill switch: the presence of a STOP_AIS.txt file in the working directory causes each process to flush and terminate gracefully on its next loop iteration.

7. Limitations and Future Work

Several acknowledged limitations bear on the accuracy of the current estimates. DWT estimation. The length-based DWT regression is the largest single source of uncertainty in the mass estimation chain. A single regression cannot capture the variation across vessel subtypes with the same length but different beam or

capacity. A planned upgrade is a decision tree or cluster-stratified regression using length, beam, and length-to-beam ratio as joint predictors. An alternative is to build an empirically observed DWT database from the system's own history, identifying full-load transits and inverting the TPCI formula.

Single-source AIS coverage. A single terrestrial AIS network produces coverage gaps, particularly for vessels at anchor in offshore terminals or beyond the line of sight of shore-based receivers. These gaps motivate the threshold relaxations described in §5.4.1 and the COG filter removal described in §5.3. The natural remedy is the addition of satellite AIS, which would provide global coverage and allow the tighter Zhang et al. thresholds and the COG filter to be reinstated.

Ballast estimation uncertainty. The ballast calculations rely on estimated DWT, estimated design draught, and MARPOL minimum ballast — all of which are approximations. The MARPOL minimum ballast formula is a regulatory floor, not an observed value, and actual ballast carried may differ. A planned improvement is to observe vessels in their ballast (light) condition — identifiable by low draught and typically a return voyage — to empirically calibrate lightship and light-ballast values for individual MMSIs.

Storage and shallow-draft events. The current model does not account for vessels used as floating storage, for which draught changes may represent inventory movements rather than cargo transfers to or from shore. Nor does it handle the practice of lightering — transferring cargo between vessels to reduce draught for transit through shallow-draft channels — which could generate draught changes without a corresponding port call. Comments in the codebase identify these as planned additions to the event classification logic.

Port coverage. The port lookup table covers 61 named locations. Vessels transacting at ports outside this list are labelled "Lost at Sea" rather than by port name. Expanding the lookup table, or replacing it with a nearest-named-port algorithm based on country of position and distance to a comprehensive port database, is a planned improvement.

8. References

Adland, R., Jia, H., & Strandenes, S. P. (2017). Are AIS-based trade volume estimates reliable? The case of crude oil exports. Maritime Policy & Management, 44(5), 657–665. https://doi.org/10.1080/03088839.2017.1309470 (https://doi.org/10.1080/03088839.2017.1309470)

AISStream.io. Real-time AIS WebSocket API. https://aisstream.io/ (https://aisstream.io/)

Danish Maritime Authority. AIS data download. https://www.dma.dk/safety-at-sea/navigational-information/download-data (https://www.dma.dk/safety-at-sea/navigational-information/download-data)

Enercón Group. Tanker definitions. https://www.enercongroup.com/definitionstanker.html (https://www.enercongroup.com/definitionstanker.html)

Equasis. Ship information search. https://www.equasis.org/EquasisWeb/restricted/ShipInfo?fs=Search (https://www.equasis.org/EquasisWeb/restricted/ShipInfo?fs=Search)

NOAA Office for Coastal Management. (2025). AIS data handler. https://coast.noaa.gov/htdata/CMSP/AISDataHandler/2025/index.html (https://coast.noaa.gov/htdata/CMSP/AISDataHandler/2025/index.html)

NOAA Office for Coastal Management. (2018). Vessel type codes [Data file]. https://coast.noaa.gov/data/marinecadastre/ais/VesselTypeCodes2018.pdf (https://coast.noaa.gov/data/marinecadastre/ais/VesselTypeCodes2018.pdf)

Warrant Group. IMO vessel codes [Data set]. GitHub. https://github.com/warrantgroup/IMO-Vessel-Codes/blob/master/data/imo-vessel-codes.csv (https://github.com/warrantgroup/IMO-Vessel-Codes/blob/master/data/imo-vessel-codes.csv)

HI Nelson. (2019). Seaports of the world [PDF]. https://www.hinelson.com/blog/wp-content/uploads/2019/09/

Seaports-of-the-World.pdf (https://www.hinelson.com/blog/wp-content/uploads/2019/09/Seaports-of-the-World.pdf)

International Maritime Organization. (2012). Amendments to the international code for the construction and equipment of ships carrying dangerous chemicals in bulk (IBC Code) (Resolution MSC.340(91)). https://wwwcdn.imo.org/localresources/en/KnowledgeCentre/IndexofIMOResolutions/MSCResolutions/MSC.340(91).pdf (https://wwwcdn.imo.org/localresources/en/KnowledgeCentre/IndexofIMOResolutions/MSCResolutions/MSC.340(91).pdf)

International Maritime Organization. (2015). Revised guidelines for the onboard operational use of shipborne automatic identification systems (AIS) (Resolution A.1106(29)). https://wwwcdn.imo.org/localresources/en/OurWork/Safety/Documents/IMO%20Documents%20related%20to/Resolution%20A.1106(29).pdf (https://wwwcdn.imo.org/localresources/en/OurWork/Safety/Documents/IMO%20Documents%20related%20to/Resolution%20A.1106(29).pdf)

Jia, H., Prakash, V., & Smith, T. (2019). Estimating vessel payloads in bulk shipping using AIS data. International Journal of Shipping and Transport Logistics, 11(1), 25–40. https://doi.org/10.1504/IJSTL.2019.096864 (https://doi.org/10.1504/IJSTL.2019.096864)

Kalokairinos, E., Mavroeidis, T., Radou, G., & Zachariou, Z. (2000–2005). Regression analysis of basic ship design values for merchant ships [Diploma theses]. National Technical University of Athens.

MARPOL Training Institute. MARPOL 73/78, Annex I, Regulation 18: Segregated ballast tanks. https://www.marpoltraininginstitute.com/MMSKOREAN/MARPOL/Annex_I/r18.htm (https://www.marpoltraininginstitute.com/MMSKOREAN/MARPOL/Annex_I/r18.htm)

Papanikolaou, A. (2014). In Ship design: Methodologies of preliminary design. Springer. https://doi.org/10.1007/978-94-017-8751-2 (https://doi.org/10.1007/978-94-017-8751-2)

Port Economics, Management and Policy. Tanker size categories. https://porteconomicsmanagement.org/pemp/contents/part5/ports-and-energy/tanker-size/ (https://porteconomicsmanagement.org/pemp/contents/part5/ports-and-energy/tanker-size/) U.S. Energy Information Administration. (2014). Tanker sizes and classes. https://www.eia.gov/todayinenergy/detail.php?id=17991 (https://www.eia.gov/todayinenergy/detail.php?id=17991)

Raymond, E. S. AIVDM/AIVDO protocol decoding: Type 8 binary broadcast message. https://gpsd.gitlab.io/gpsd/AIVDM.html#_type_8_binary_broadcast_message (https://gpsd.gitlab.io/gpsd/AIVDM.html#_type_8_binary_broadcast_message)

Schneekluth, H., & Bertram, V. (1998). Ship design for efficiency and economy (2nd ed.). Butterworth-Heinemann.

Zhang, R., Dong, D., Chen, X., Zhang, B., Zhang, Y., Ye, L., Liu, B., Zhao, Y., & Peng, C. (2025). AIS data-driven analysis for identifying cargo handling events in international trade tankers. Ocean Engineering, 317, Article 120016. https://doi.org/10.1016/j.oceaneng.2024.120016 (https://doi.org/10.1016/j.oceaneng.2024.120016)